EE 66 Signals, Dynamics, and Information
Fall 2026 Note 9

9.1 Overview and Motivation

PCA is a tool for data analysis. When we collect data, there are lots of factors that influence the values we measure. Our goal is to figure out the most important ones. This allows us to compress our data by removing the dimensions ("factors") that are not important. We do this via *principal component analysis* (PCA).

9.2 Principal Component Analysis

Suppose we have collected some noisy data points, which are represented as vectors $\mathbf{x}_1, \ldots, \mathbf{x}_n \in \mathbb{R}^d$. Let

$$\mathbf{X} := \begin{bmatrix} \mathbf{x}_1 & \cdots & \mathbf{x}_n \end{bmatrix}, \tag{1}$$

be the so-called *data matrix*, i.e., arranging the data points as the *columns* of $\mathbf{X}$.

The first step in PCA is always to center the data around its mean $\frac{1}{n} \cdot \mathbf{X}\mathbf{1}_n$. That is, we replace $\mathbf{X}$, our uncentered data matrix, with $\mathbf{A} := \mathbf{X}(\mathbf{I}_n - \frac{1}{n}\mathbf{1}_n\mathbf{1}_n^\top)$. More concretely, we subtract the column mean from each data point in $\mathbf{X}$ (note: the symbol $\mathbf{1}_n \in \mathbf{R}^{n\times 1}$ is a vector with all elements being 1) to get the *demeaned* data matrix $\mathbf{A}$. From now on, we will refer to this demeaned data matrix.

Suppose $\text{rank}(\mathbf{A}) = r \leq \min\{n, d\}$. Further suppose the so-called "*ground truth*" data (i.e., data without the noise) actually would span a ℓ-dimensional subspace $\mathbf{S}_{\text{gt}}$ of $\mathbb{R}^d$, with $\ell \leq r$. The remaining $r - \ell$ dimensions, in this case, would be the product of noise. The goal of principal component analysis (PCA) is to find, from the noisy data, the relevant ℓ-dimensional subspace $\mathbf{S}_{\text{gt}}$ which the dataset spans.

Suppose $\text{rank}(\mathbf{A}) = r \leq \min\{n, d\}$.

Consider the following plot:

UCB EE 66, Fall 2026, Note 9, All Rights Reserved. This may not be publicly shared without explicit permission.

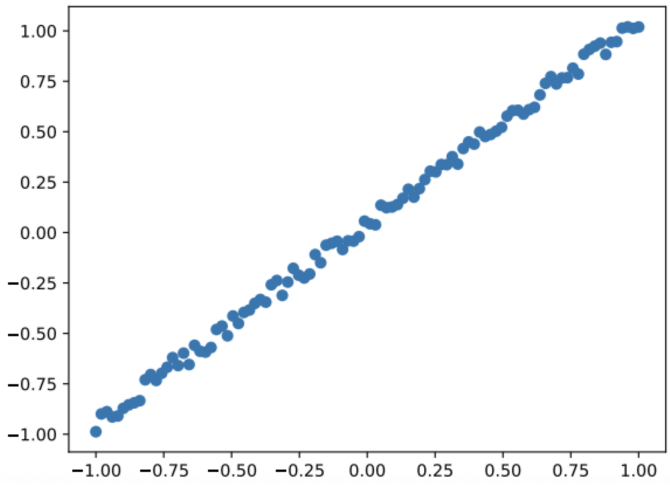


Figure 1: Example "Noisy" Data

Clearly, the data is very linear in the sense that the first coordinate is approximately equal to the second coordinate. However, due to noise, the data we have must be represented in 2D. However, the extra dimension here is unnecessary and requires extra memory to store on top of that. This leads to the natural problem of attempting to reduce the amount of memory needed and to only consider the linearity of the data (we don't care about the noise).

To solve this, we could choose to *project* the data points into a one dimensional space (so the data is forced to be linear); this is fundamentally what PCA does. PCA projects data points into lower dimension spaces to (1) remove extraneous dimensions and (2) describe data points with less memory.

We know we want to project onto some subspace, but what subspace should we project onto? The best subspace attempts to *minimize* the distances to each data point. Mathematically, if we want to project data points $\mathbf{a}_i \in \mathbb{R}^d$ into a subspace $\mathbf{S}$ of maximum dimension ℓ, the best subspace $\mathbf{S}_{\text{PCA}}$ is:

$$\mathbf{S}_{\text{PCA}} = \arg\min_{\mathbf{S} \subseteq \mathbb{R}^d} \sum_{i=1}^{n} \|\mathbf{a}_i - \text{proj}_{\mathbf{S}}(\mathbf{a}_i)\|^2 \tag{2}$$

We call $\sum_{i=1}^{n} \|\mathbf{a}_i - \text{proj}_{\mathbf{S}}(\mathbf{a}_i)\|^2$ the *objective function* for this optimization problem.

For some intuition, suppose we had 4 points in $\mathbb{R}^2$. This objective function would be the sum of the squared lengths of the red lines in this picture, where the subspace $\mathbf{S}$ is a black line and the data points $\mathbf{a}_1, \ldots, \mathbf{a}_4$ are blue dots:

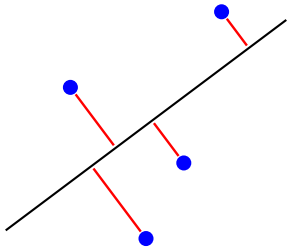


Figure 2: Objective function visualization for PCA.

Contrast this to the least squares objective function, which computes the sum of squares of the *vertical* residuals (instead of the orthogonal residuals):

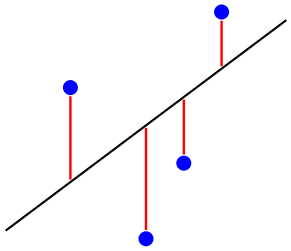


Figure 3: Objective function visualization for least squares.

Now, we explore how to compute this objective function. If $\mathbf{W} \in \mathbb{R}^{d \times \ell}$ has orthonormal columns which span $\mathbf{S}$, i.e., $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_\ell$ and $\text{Col}(\mathbf{W}) = \mathbf{S}$, then

$$\text{proj}_{\mathbf{S}}(\mathbf{a}_i) = \mathbf{W}\mathbf{W}^\top \mathbf{a}_i. \tag{3}$$

The above equation follows directly from the least squares derivation of a projection matrix in the case in which the columns of this projection matrix are orthonormal.

$$\mathbf{Q}(\mathbf{Q}^\top \mathbf{Q})^{-1}\mathbf{Q}^\top = \mathbf{Q}\mathbf{Q}^\top \tag{4}$$

which means that the matrix $\mathbf{Q}\mathbf{Q}^\top$ projects a vector onto $\text{Col}(\mathbf{Q})$.

This means that our objective function takes the form

$$\sum_{i=1}^{n} \left\| \mathbf{a}_i - \mathbf{W}\mathbf{W}^\top \mathbf{a}_i \right\|^2. \tag{5}$$

And we would try to minimize this over subspaces $\mathbf{S}$, i.e., over matrices $\mathbf{W} \in \mathbb{R}^{d \times \ell}$ with orthonormal columns such that $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_\ell$. This leads to the following optimization problem, which can be solved via the SVD.

Theorem 9.1: Principal Component Analysis

Let $\mathbf{a}_1, \ldots, \mathbf{a}_n \in \mathbb{R}^d$ be normalized data points. If $\mathbf{A} = \begin{bmatrix} \mathbf{a}_1 & \cdots & \mathbf{a}_n \end{bmatrix}$ is the data matrix, with SVD $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$, then

$$\mathbf{U}_\ell \in \underset{\mathbf{W} \in \mathbb{R}^{d \times \ell}}{\arg\min} \quad \sum_{i=1}^{n} \left\| \mathbf{a}_i - \mathbf{W}\mathbf{W}^\top \mathbf{a}_i \right\|^2 \tag{6}$$

$$\text{s.t.} \quad \mathbf{W}^\top \mathbf{W} = \mathbf{I}_\ell, \tag{7}$$

where $\mathbf{U}_\ell = \begin{bmatrix} \mathbf{u}_1 & \cdots & \mathbf{u}_\ell \end{bmatrix}$ is the first ℓ columns of $\mathbf{U}$. Theese vectors are also called the *principal components* of $\mathbf{A}$ in that order. (For instance, $\mathbf{u}_1$ is the first principal component, $\mathbf{u}_2$ is the second principal component, etc.) *See Appendix A for the proof.*

Based on our knowledge of how $\mathbf{u}_i$'s are the eigenvectors of $\mathbf{A}\mathbf{A}^\top$, we have the following equivalent, alternate characterization, which tells us how to calculate the principal components without calculating the SVD.

Theorem 9.2: Principal Components as Eigenvectors: Let $\mathbf{A} = \begin{bmatrix} \mathbf{a}_1 & \cdots & \mathbf{a}_n \end{bmatrix} \in \mathbb{R}^{d \times n}$ be a data matrix. The principal components of $\mathbf{A}$ are the eigenvectors of $\mathbf{A}\mathbf{A}^\top$, ordered in non-increasing order by the value of the corresponding eigenvalue, with ties broken arbitrarily[1].

To give an idea of how effective this procedure is, suppose we have the following centered data (blue), with the "ground truth" subspace $\mathbf{S}_{\text{gt}}$ (green) also shown.

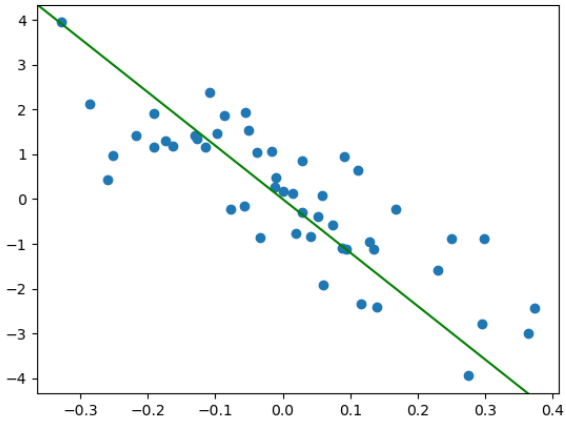


Figure 4: A sample dataset with $n = 50$, $d = 2$, and $\dim(\mathbf{S}_{\text{gt}}) = 1$ (green).

If we also plot the first principal component subspace (shown in orange), we get the following plot.

[1] By ties, we mean cases in which two or more eigenvalues in the non-increasing sequence are exactly equal.

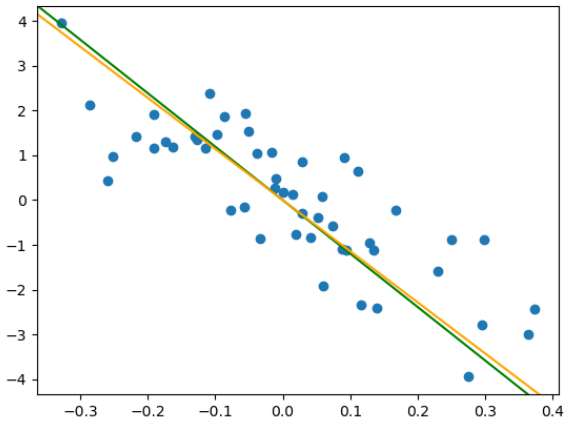


Figure 5: A sample dataset with $n = 50$, $d = 2$, $\dim(\mathbf{S}_{\text{gt}}) = 1$ (green), and $\dim(\mathbf{S}_{\text{PCA}}) = 1$ (orange).

The idea is: *the principal components learned from data are very similar to the true low-rank subspace the ground truth data lie on!* This idea generalizes to many dimensions as well.

9.2.1 Column Data vs. Row Data

In many contexts, data will be stored in the *rows* of the data matrix instead of the columns as such:

$$\mathbf{B} = \begin{bmatrix} \mathbf{b}_1^\top \\ \mathbf{b}_2^\top \\ \vdots \\ \mathbf{b}_d^\top \end{bmatrix} \tag{8}$$

Suppose the SVD of $\mathbf{B}$ is $\mathbf{U}_r\Sigma_r\mathbf{V}_r^\top$. To use PCA with a data matrix like this, the process is identical to before but the principal component vectors are $\mathbf{v}_i$ (column vectors of $\mathbf{V}_r$ from the SVD) instead of $\mathbf{u}_i$.

This should hopefully make sense. One way to think about it is that just like $\text{Col}(\mathbf{B}) = \text{Col}(\mathbf{U_r})$, $\text{Row}(\mathbf{X}) = \text{Col}(\mathbf{V_r})$ so the column vectors of $\mathbf{V}_r$ (which are $\mathbf{v}_i$) describe the row vectors of the matrix $\mathbf{B}$.

Another equivalent approach is that we could first calculate $\mathbf{B}^\top = \begin{bmatrix} \mathbf{b}_1 & \mathbf{b}_2 & \cdots & \mathbf{b}_d \end{bmatrix}$, which has SVD:

$$\mathbf{B}^\top = (\mathbf{U}_r\Sigma_r\mathbf{V}_r^\top)^\top = \mathbf{V}_r\Sigma_r\mathbf{U}_r^\top \tag{9}$$

Since the data points are now column vectors of $\mathbf{B}^\top$, and $\mathbf{V}_r$ has replaced the position of $\mathbf{U}_r$ in the SVD, it makes sense that $\mathbf{v}_i$ are the principal component vectors for this data setup.

For the rest of the sections, we will use the column data view since it is easier to formulate results using it, but note that the results can be easily transferred to row view data by the above statements.

9.3 Denoising and Dimensionality Reduction

Suppose we have a dataset $\mathbf{A} \in \mathbb{R}^{d \times n}$ (where the data points are columns of $\mathbf{A}$) on which we perform PCA, and get ℓ principal components $\mathbf{u}_1, \ldots, \mathbf{u}_\ell$. Suppose we are given a new noisy vector $\mathbf{a} \in \mathbb{R}^d$, and we know that the "ground truth" de-noised vector approximately lies in our low-rank ℓ-dimensional subspace. To recover an estimate for the original ground truth vector, we project onto this subspace, i.e., our principal component subspace. From what we know about projections, the formula to project onto $\text{Span}(\mathbf{u}_1, \ldots, \mathbf{u}_\ell) = \text{Col}(\mathbf{U}_\ell)$ is

$$\text{proj}_{\text{Span}(\mathbf{u}_1,\ldots,\mathbf{u}_\ell)}(\mathbf{a}) = \text{proj}_{\text{Col}(\mathbf{U}_\ell)}(\mathbf{a}) = \mathbf{U}_\ell \mathbf{U}_\ell^\top \mathbf{a}. \tag{10}$$

Since this process removes the residual noise and preserves the essential low-rank structure of $\mathbf{a}$, we call it *denoising*.

We may also discuss PCA from the viewpoint of *data compression* and *dimensionality reduction*. Namely, if $\ell \leq d$, then we can approximately represent points $\mathbf{a}$ in $\mathbb{R}^d$ in our dataset by vectors $\mathbf{w}$ in $\mathbb{R}^\ell$; these vectors $\mathbf{w}$ in $\mathbb{R}^\ell$ precisely store the coefficients in the linear combination of $\mathbf{u}_1, \ldots, \mathbf{u}_\ell$ required to approximately generate $\mathbf{a}$. The closest point in the ℓ-dimensional principal component subspace to $\mathbf{a}$ is its projection, which has the formula

$$\text{proj}_{\text{Span}(\mathbf{u}_1,\ldots,\mathbf{u}_\ell)}(\mathbf{a}) = \mathbf{U}_\ell \mathbf{U}_\ell^\top \mathbf{a} = \sum_{i=1}^{\ell} \langle \mathbf{a}, \mathbf{u}_i \rangle \mathbf{u}_i. \tag{11}$$

Then the coefficients w_i which make up our compressed vector $\mathbf{w}$ are precisely the inner products $\langle \mathbf{a}, \mathbf{u}_i \rangle$, so we have

$$\mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_\ell \end{bmatrix} = \begin{bmatrix} \langle \mathbf{a}, \mathbf{u}_1 \rangle \\ \vdots \\ \langle \mathbf{a}, \mathbf{u}_\ell \rangle \end{bmatrix} = \begin{bmatrix} \mathbf{u}_1^\top \mathbf{a} \\ \vdots \\ \mathbf{u}_\ell^\top \mathbf{a} \end{bmatrix} = \begin{bmatrix} \mathbf{u}_1^\top \\ \vdots \\ \mathbf{u}_\ell^\top \end{bmatrix} \mathbf{a} = \mathbf{U}_\ell^\top \mathbf{a}. \tag{12}$$

To summarize, we can represent a vector $\mathbf{a} \in \mathbb{R}^d$ in our ℓ-dimensional subspace by ℓ coordinates—that is, a vector $\mathbf{U}_\ell^\top \mathbf{a}$ in $\mathbb{R}^\ell$ whose entries are the inner products of $\mathbf{a}$ with the first ℓ principal components. Even if $\mathbf{a}$ is not in our ℓ-dimensional subspace, we can represent $\mathbf{a}$ by a vector in $\mathbb{R}^\ell$ which approximately generates $\mathbf{a}$ (as opposed to exactly generating $\mathbf{a}$). To recover the original vector (or in the latter case, an estimate for it), we can just multiply by $\mathbf{U}_\ell$ again, which reduces to denoising the data.

For row data, everything above is the same except we replace $\mathbf{U}_\ell$ with $\mathbf{V}_\ell$.

If $\ell \ll d$ then this is a big success for us, since we have compressed our data a lot, and distilled it to its most important directions of variation.

9.4 Picking the Best Number of Principal Components

So far, we have assumed that we know the true dimension of the data points. For example, recall the noisy linear example from Figure 1. We knew that the underlying data was linear (1-dimensional) visually. However, for higher dimensional data, unless we have knowledge of the underlying data, we will not know the true dimension of the data. How do we choose what dimension subspace to project into using PCA in this case?

You will study this more in machine learning contexts (such as in CS 189), but we can determine the best dimension by splitting our data into *training* and *test* datasets. We choose some dimension ℓ to project into, perform the projection, and use the test dataset to determine if the projection is good or not. We can repeat this for different values of ℓ to find out which one is the best. This procedure is known as hyperparameter tuning.

9.5 PCA Intuition/Summary

In this note, we talked a lot about the math behind PCA and its various uses in optimization and machine learning contexts. These will be covered more in depth in upper division classes like EECS 127 and CS 189.

The most important things to understand about PCA currently are the basics (ex. what principal component vectors are, how to find them with the SVD, what PCA is doing visually/graphically). In this section, we will try to summarize this without too much math.

Suppose we have some *data vectors* $\mathbf{x}_1, \ldots, \mathbf{x}_d$ and place them as the column vectors in a *data matrix*:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 & \cdots & \mathbf{x}_d \end{bmatrix} \tag{13}$$

For simplicity, assume the mean of the data vectors is $\mathbf{0}$ (if not, you would want to subtract each column of $\mathbf{X}$ by the mean before calculating PCA).

The SVD of $\mathbf{X}$, expressed in all three forms, is:

$$\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^\top = \mathbf{U}_r\Sigma_r\mathbf{V}_r^\top = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top \tag{14}$$

The vectors $\mathbf{u}_i$ are called the *principal component vectors* of the data matrix $\mathbf{X}$.

Why does this make sense? Well, from SVD, we know that $\text{Col}(\mathbf{X}) = \text{Col}(\mathbf{U_r}) = \text{span}(\{\mathbf{u}_1, \ldots, \mathbf{u}_r\})$. Thus, the $\mathbf{u}_i$ vectors describe the columns of $\mathbf{X}$, which are the data points $\mathbf{x}_i$.

The order of the principal component vectors matters: the *first* principal component vector, $\mathbf{u}_1$, is the most important principal component vector. $\mathbf{u}_2$ is the second most important, etc. By *most important*, we mean two equivalent points:

- Projecting the data points $\mathbf{x}_i$ onto $\mathbf{u}_1$ results in the least total error between the actual data vectors and their projections compared to projecting onto any other 1-dimensional space.
- Projecting the data points $\mathbf{x}_i$ onto $\mathbf{u}_1$ results in the maximum variance between the projections compared to projecting onto any other 1-dimensional space. (You can think of the variance of a feature as the level of impact that a feature has on the data. Higher variance means the feature has a larger effect on what the data looks like and we want to capture the features that best describe the data.)

Why does this make sense? Well, we know that the singular values σ_i are ordered in *descending* order ($\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r$) so the outer product $\sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top$ is the *most prevalent* in the matrix $\mathbf{X}$. Thus, $\mathbf{u}_1$ is the most important vector (the first principal component) to describe the column vectors of $\mathbf{X}$.

We can extend this to projections into ℓ-dimensional spaces (with $\ell \leq d$) instead of just 1-dimensional space by projecting onto a matrix consisting of the first ℓ principal components of $\mathbf{X}$, $\mathbf{U}_\ell = \begin{bmatrix} \mathbf{u}_1 & \cdots & \mathbf{u}_l \end{bmatrix}$:

- Projecting the data points $\mathbf{x}_i$ onto $\mathbf{U}_\ell$ results in the least total error between the actual data vectors and their projections compared to projecting onto any other ℓ-dimensional space.
- Projecting the data points $\mathbf{x}_i$ onto $\mathbf{U}_\ell$ for results in the maximum variance between the projections compared to projecting onto any other ℓ-dimensional space.

In both cases, projecting onto the PCA vectors, in order of importance, helps keep data points close to their original positions (and thus distinctly separate from other points); the least amount of data information is lost from these projections.

Projections onto principal components involve two different parts.

The *projection* of a data point $\mathbf{y}$ into ℓ-dimensional space is performed with the following calculation:

$$\mathbf{y}_\ell = \mathbf{U}_\ell^\top \mathbf{y} = \begin{bmatrix} \mathbf{u}_1^\top \mathbf{y} \\ \vdots \\ \mathbf{u}_\ell^\top \mathbf{y} \end{bmatrix} \tag{15}$$

Notice how the output vector is ℓ-dimensional and consists of the *coefficients* of the projections of $\mathbf{y}$ onto each of the chosen ℓ principal component vectors.

The *reconstruction* of the projected data point $\mathbf{y}_\ell$ back into d-dimensional space is performed with the following calculation:

$$\mathbf{y}_{\text{recontructed}} = \mathbf{U}_\ell \mathbf{y}_l = \mathbf{U}_\ell \mathbf{U}_\ell^\top \mathbf{y} = \begin{bmatrix} \mathbf{u}_1 \mathbf{u}_1^\top \mathbf{y} \\ \vdots \\ \mathbf{u}_\ell \mathbf{u}_\ell^\top \mathbf{y} \end{bmatrix} \tag{16}$$

Note that this is closer to the usual definition of projection that we use: $\mathbf{U}_\ell \mathbf{U}_\ell^\top$ is the projection matrix that projects the d-dimensional data vector $\mathbf{y}$ to another d-dimensional vector $\mathbf{y}_{\text{reconstructed}} \in \text{Col}(\mathbf{U}_\ell)$ (which consists of the projections onto each of the principal component vectors).

Using $\mathbf{U}_\ell^\top$ by itself simply extracts the coefficients of the projections onto the principal component vectors (this is often very useful by itself since it is effectively a more compact version of the full data point).

If we are using row data instead of column data, i.e.

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_d^\top \end{bmatrix} \tag{17}$$

Then all the statements above are the same except the principal component vectors are $\mathbf{v}_i$ instead of $\mathbf{u}_i$ and we replace $\mathbf{U}_\ell$ with $\mathbf{V}_\ell$ accordingly.

A Proof of Theorem 9.1

Proof of 9.1. Assume that all of the data vectors $\mathbf{x}_i$ already have mean 0. We first simplify each term in the objective function.

$$\left\|\mathbf{x}_i - \mathbf{W}\mathbf{W}^\top \mathbf{x}_i\right\|^2 = \langle \mathbf{x}_i - \mathbf{W}\mathbf{W}^\top \mathbf{x}_i, \mathbf{x}_i - \mathbf{W}\mathbf{W}^\top \mathbf{x}_i \rangle \tag{18}$$

$$= \langle \mathbf{x}_i, \mathbf{x}_i \rangle - 2\langle \mathbf{W}\mathbf{W}^\top \mathbf{x}_i, \mathbf{x}_i \rangle + \langle \mathbf{W}\mathbf{W}^\top \mathbf{x}_i, \mathbf{W}\mathbf{W}^\top \mathbf{x}_i \rangle \tag{19}$$

$$= \|\mathbf{x}_i\|^2 - \left\|\mathbf{W}^\top \mathbf{x}_i\right\|^2. \tag{20}$$

Thus, we can reduce the optimization of the objective to the simpler optimization

$$\operatorname*{arg\,min}_{\substack{\mathbf{W}\in\mathbb{R}^{n\times \ell} \\ \mathbf{W}^\top\mathbf{W}=\mathbf{I}_\ell}} \sum_{i=1}^n \left\|\mathbf{x}_i - \mathbf{W}\mathbf{W}^\top \mathbf{x}_i\right\|^2 = \operatorname*{argmax}_{\substack{\mathbf{W}\in\mathbb{R}^{n\times \ell} \\ \mathbf{W}^\top\mathbf{W}=\mathbf{I}_\ell}} \sum_{i=1}^n \left\|\mathbf{W}^\top \mathbf{x}_i\right\|^2. \tag{21}$$

Now expanding the squared norm in terms of the sum of squares of each entry, we have

$$\sum_{i=1}^n \left\|\mathbf{W}^\top \mathbf{x}_i\right\|^2 = \sum_{i=1}^n \sum_{k=1}^\ell (\mathbf{w}_k^\top \mathbf{x}_i)^2 \tag{22}$$

$$= \sum_{k=1}^\ell \mathbf{w}_k^\top \left(\sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top\right) \mathbf{w}_k \tag{23}$$

$$= \sum_{k=1}^\ell \mathbf{w}_k^\top \left(\mathbf{A}\mathbf{A}^\top\right) \mathbf{w}_k. \tag{24}$$

Computing $\mathbf{A}\mathbf{A}^\top = \mathbf{U}\Sigma\Sigma^\top\mathbf{U}^\top$, we have

$$\sum_{i=1}^n \left\|\mathbf{W}^\top \mathbf{x}_i\right\|^2 = \sum_{k=1}^\ell \mathbf{w}_k^\top \mathbf{U}\Sigma\Sigma^\top\mathbf{U}^\top \mathbf{w}_k. \tag{25}$$

We introduce the change of coordinates $\widetilde{\mathbf{w}}_k := \mathbf{U}^\top \mathbf{w}_k$. Note that since the $\mathbf{w}_k$ are orthonormal, so too are the $\widetilde{\mathbf{w}}_k$. Expanding out $\Sigma\Sigma^\top$, we have

$$\sum_{i=1}^n \left\|\mathbf{W}^\top \mathbf{x}_i\right\|^2 = \sum_{k=1}^\ell \widetilde{\mathbf{w}}_k^\top \Sigma\Sigma^\top \widetilde{\mathbf{w}}_k. \tag{26}$$

Since the $\widetilde{\mathbf{w}}_k$ are orthonormal, a maximizing choice of $\widetilde{\mathbf{w}}_1, \ldots, \widetilde{\mathbf{w}}_\ell$ is $\mathbf{e}_1, \ldots, \mathbf{e}_\ell$ (i.e., the standard basis vectors), at which point the objective value is $\sum_{k=1}^\ell \sigma_k^2$. The corresponding $\mathbf{w}_k$ is given by

$$\mathbf{w}_k = \mathbf{U}\widetilde{\mathbf{w}}_k = \mathbf{U}\mathbf{e}_k = \mathbf{u}_k, \tag{27}$$

so a maximizing $\mathbf{W}$ is $\mathbf{U}_\ell$ as desired. □